Image Classification with Neural Networks: A Mathematical Approach to Flower Recognition

Emmalyn Holmquist
Advisor: Dr. Kristin Kuter

18 March, 2026

Contents

1 Introduction **2**

2 Feeding Data into a Neural Network: Understanding Tensors **2**

3 Architecture and Terminology of a Basic Neural Network **6**

3.1 Neural Networks: High Level 7

3.2 Neural Network Notation 9

3.3 Parameter Representation 9

4 Extracting Features through the Forward Pass **10**

4.1 Neural Networks as a Series of Nested Equations 10

4.2 Importance of σ 11

4.3 Bringing it All Together 12

5 Training the Model: Beginning the Backward Pass with Cost **12**

5.1 Understanding Cost Function & Activation Function Pairings 13

6 Continuing the Learning Process with Backpropagation **14**

6.1 Equation BP1 15

6.2 Equation BP2 16

6.3 Equation BP3 18

6.4 Equation BP4 19

6.5 Calculating the Gradient of Binary Cross Entropy with Sigmoid 20

6.5.1 Deriving $\frac{\partial C}{\partial \hat{y}}$ 21

6.5.2 Deriving $\frac{\partial \hat{y}}{\partial z^L}$ 22

6.5.3 Deriving the Final Error of the Network 22

7 Gradient Descent: the Update Process **23**

7.1 Motivation 23

7.2 Defining Gradient Descent 24

7.3 Improving Gradient Descent 24

8 Application: Using a Neural Network for Image Classification **25**

9 Conclusion **26**

A Appendix: Definitions and Notation **28**

A.1 Key Definitions 28

A.2 Notation 28

A.3 Core Equations 29

References **31**

1 Introduction

Neural networks play a critical role in modern machine learning by enabling the modeling of complex, non-linear relationships within data. These models approximate functions by composing layers of linear transformations with non-linear activation functions, allowing them to learn patterns directly from high-dimensional inputs such as images. Rather than relying on manually engineered features, neural networks automatically extract meaningful representations and capture intricate structures present in the data. In this paper, we explore the mathematical foundations and computational implementation of neural networks for image classification. We examine how neural networks can be constructed as compositions of nested functions, represented using precise mathematical notation, and utilize tensors for data representation. A central focus is the derivation of the backpropagation algorithm, where gradients are computed efficiently through repeated application of the chain rule. We analyze the role of activation functions and cost functions in the learning process, including the pairing of sigmoid activation with binary cross-entropy loss, and show how this leads to a simplified gradient expression. Using these principles, we design and implement a neural network in Python to classify images of flowers into distinct categories. The model is trained using gradient descent to iteratively update its parameters and minimize classification error. Through this implementation, we develop a deeper understanding of how neural networks learn from visual data and their role in solving image classification problems.

2 Feeding Data into a Neural Network: Understanding Tensors

In order to understand how a neural network works, we first have to develop the idea of a tensor, which is the key to feeding data into a neural network. There are two ways that we may use tensors: in mathematics, they are simply considered high dimensional matrices (or a matrix of matrices) while in computer science we refer to them as a tool to feed data into a neural network

using Python. In Python, tensors are represented as multi-dimensional NumPy arrays, including scalars (rank-0), vectors (rank-1), matrices (rank-2), cubes (rank-3), and more. Further, we define a tensor by giving its rank/number of axes, data type, and shape.

Definition 2.1. In machine learning, a **tensor** is a multi-dimensional array defined by giving its rank, data type, and shape.

In terms of data representation, tensors of different ranks are capable of representing data of different forms. See the table below for standard data and their corresponding tensor representations.

Data	Rank	Standard Representation
Vector	2	(Samples, Features)
Time-series/Sequence	3	(Samples, Timesteps, Features)
Image	4	(Samples, Height, Width, Channels)
Video	5	(Samples, Frames, Height, Width, Channels)

Figure 1: Common Data & Tensor Representations

Consider a dataset consisting of four color images, each with dimensions 24×24 pixels. Because the images are in color, each one contains three channels corresponding to the red, green, and blue (RGB) values. If the images were instead grayscale, they would only have a single channel (representing the grayscale) and the data could be represented as a rank-3 tensor with shape (4, 24, 24). However, for color images, the data is represented as a rank-4 tensor with shape (4, 24, 24, 3), due to having multiple channels. In these tensors, each axis corresponds to a specific dimension: the number of samples in the dataset, the height of the images, the width of the images, and the number of color channels for the images. The values in this tensor are typically stored as floating-point numbers, representing the saturation of each color channel at every pixel. Further, the scale that has been commonly established to represent pixel color concentrations has been noted on a scale of 0 to 255 for each saturation of red, green, and blue (or a scale of 0 to 255 for the grayscale).

Figure 2: 2x2 Pixel Grayscale Images

However, when we implement this with code, we typically scale the values to be between 0 and 1, to improve computational efficiency (as described in the Application section).

Example 2.2. We can represent images data as tensors more concretely with an example of two, 2×2 pixel grayscale images. Consider Figure 2, for example. The shape of our tensor would be given by (2, 2, 2), of data type float. We can then represent the data of the images in a matrix of matrices, where each matrix inside the larger matrix represents an image, and we indicate a 0 or 255 for the presence of a black or white pixel, respectively. We can visualize the contents of our tensor as given by A, below:

$$A = \left[\begin{pmatrix} 0 & 255 \\ 255 & 0 \end{pmatrix}, \quad \begin{pmatrix} 0 & 0 \\ 255 & 0 \end{pmatrix} \right].$$

Example 2.3. Further, we might represent two colorized, 2×2 pixel images in matrix form. When visualizing colorized images, since our number of axes is 4 instead of 3 (we move from 1 to 3 channels, thus our features become more complex) their matrix representation becomes additionally

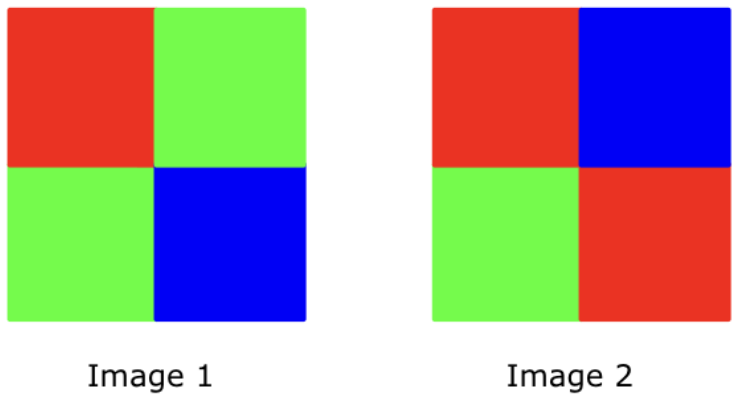

Figure 3: 2x2 Pixel Colorized Pixel Images

more complex. Let's start with visualizing just one image:

$$A_i = \begin{bmatrix} (R_i) & (G_i) & (B_i) \end{bmatrix}.$$

Since colorized images have three channels, we thus need to represent them using three matrices per image. A_i represents the matrix representation of any particular colorized image i, where matrix R denotes the presence of a red pixel, matrix G denotes the presence of a green pixel, and matrix B denotes the presence of a blue pixel. The size of these matrices is variable to the size of the image, where the number of rows in the matrix is the height of the image (number of pixels) and the number of columns of the matrix is the width of the image (number of pixels). Consider the matrix representation for image 1 of Figure 3:

$$A_1 = \begin{bmatrix} \begin{pmatrix} 255 & 0 \\ 0 & 0 \end{pmatrix} & \begin{pmatrix} 0 & 255 \\ 255 & 0 \end{pmatrix} & \begin{pmatrix} 0 & 0 \\ 0 & 255 \end{pmatrix} \end{bmatrix}.$$

Here, A_1 denotes the matrix representation of image 1, where a 0 represents the absence of that color in that pixel, and 255 represents a 100% concentration of that color in the pixel, following the format of A_i defined above.

Further, Image 2 is defined as:

$$A_2 = \begin{bmatrix} \begin{pmatrix} 255 & 0 \\ 0 & 255 \end{pmatrix} & \begin{pmatrix} 0 & 0 \\ 255 & 0 \end{pmatrix} & \begin{pmatrix} 0 & 255 \\ 0 & 0 \end{pmatrix} \end{bmatrix}.$$

Finally, the collection of them together in a tensor would be in the form:

$$A = \begin{bmatrix} (A_1) & (A_2) \end{bmatrix}.$$

In Python, data is fed into neural networks using tensors because they are built on highly optimized NumPy arrays, allowing complex matrix operations to be performed efficiently. This efficiency becomes especially important when working with large datasets, where tensors can grow significantly in size and require numerous computations. Ultimately, using tensors is essential for maintaining strong computational performance at scale. Additionally, frameworks like Keras and TensorFlow are designed specifically to work with tensors, providing a wide range of built-in operations that eliminate the need to implement neural networks from scratch.

3 Architecture and Terminology of a Basic Neural Network

Once we understand how data is fed into a neural network, we can begin to understand the structure and operations behind it. The diagram in Figure 4 below illustrates the basic structure of a neural network, where each "column" of neurons represents a layer. The **input layer** is where we feed in the data, where for a grayscale image, each neuron might represent a pixel in the image. For example, for one of the 2×2 pixel, black and white images from Figure 2, the neural network in Figure 4 would fit the dimensions needed to input the image. If we were to input one of the 2×2 pixel colorized images from Figure 3, then we would increase the number of our input layer neurons from 4 to 12 (three neurons for each pixel, where the three neurons represent the value of red, green, or blue in the image). Further, the **hidden layers** are highly customizable, and can hold any number of neurons, although typically reduce in size until the **output layer**, where for

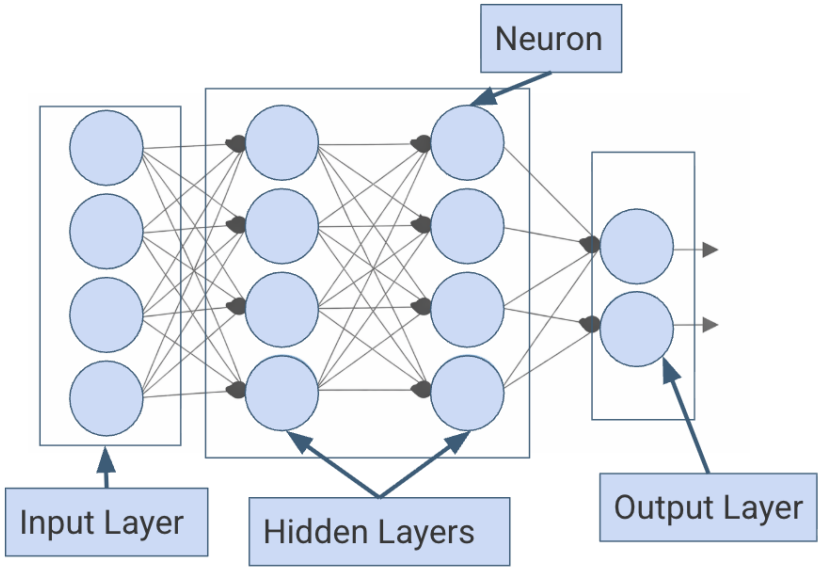

Figure 4: Diagram of Neural Network

image classification the number of neurons often represents the number of classes. In Figure 4, the neural network can be used to predict two classes (since it has two output layer neurons).

3.1 Neural Networks: High Level

In terms of a neural network, we can broadly understand the process of extracting information from the data as the early layers detecting pixels, the middle layers detecting the parts of an image, and the final layer using this knowledge to predict. While this may not be mathematically very accurate to what the neural network is doing, it is an intuitive way to understand the process of a neural network. See Figure 5 for a visualization of this process.

Further, each neuron in the neural network has its own **activation** and **bias**, and each connected edge in the neural network diagram represents a **weight**.

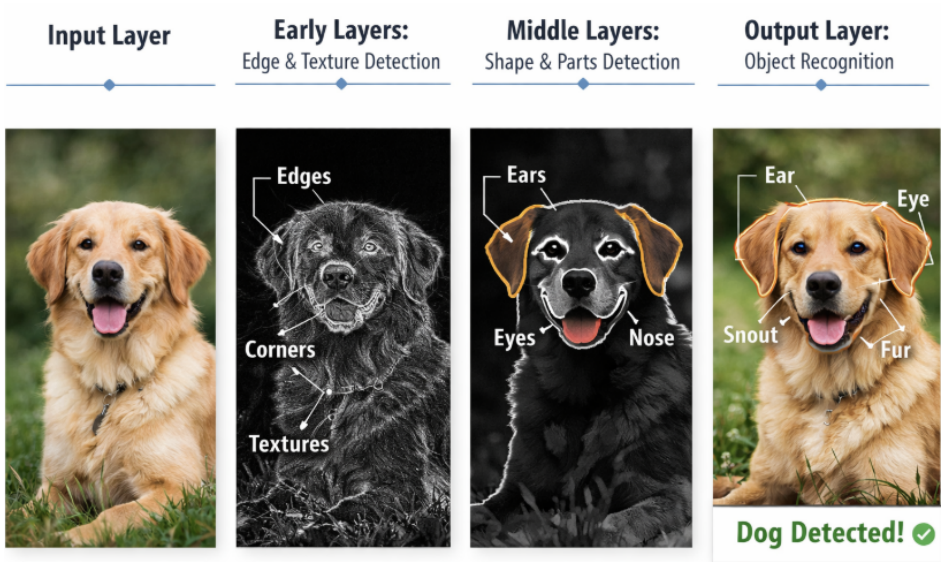

Figure 5: Intuitive Example of the Process of a Neural Network

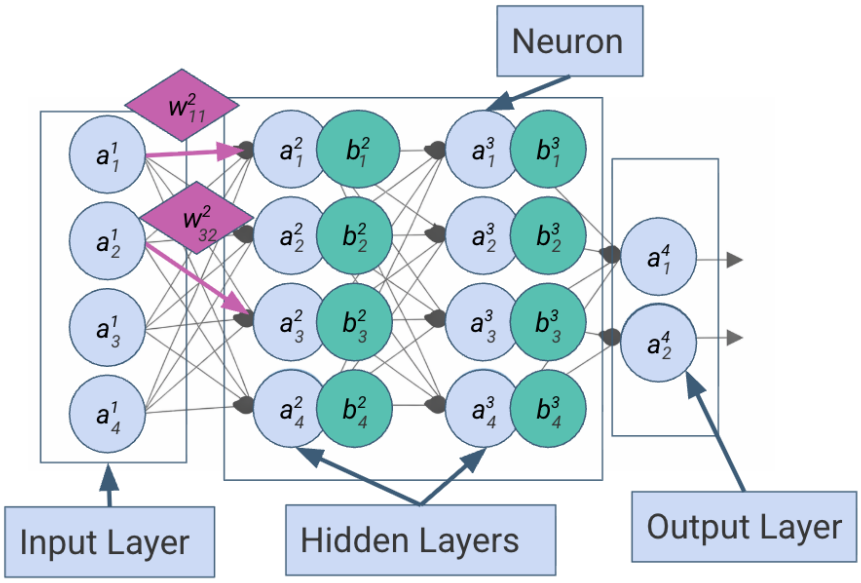

Figure 6: Weights, Biases, and Activations in a Neural Network

3.2 Neural Network Notation

The notation used to describe a neural network can be tedious, but it is essential for understanding how the model makes predictions mathematically. We rely on this notation to express the sequence of nested equations that define the network's computations. As illustrated in Figure 6, the superscript on each weight, activation, or bias indicates the layer to which that parameter belongs. The meaning of the subscript, however, differs depending on the parameter. For activations and biases, the subscript identifies the specific neuron within a given layer. For weights, the subscript denotes the connection between neurons, specifying both the source neuron and the destination neuron. For example, the weight w_{12}^2 represents the weight of layer 2 from neuron 2 in the first layer to neuron 1 in the second layer.

3.3 Parameter Representation

An important thing to note at this stage is that the values of weights, activations, and biases are singular values, which then get grouped into matrices and vectors, to be represented in equations. Activations and biases are most often represented as a vector for each layer, while weights are most commonly represented as a matrix, where the index of the columns of the matrix represent the neuron that the weight is coming from in the previous layer, and the index of the rows of the weight matrix represent the neuron of the next layer that the weight is going to. All of the neurons from the previous layer are connected to each individual neuron of the next layer by weights. Thus, we could represent the weights to a single neuron with a vector or use a weight matrix to represent all of the connections to each neuron in the next layer.

In general, consider the weight matrix W^l below, where the superscript l represents a specific layer l. On weight w_{jk}^l, subscript j represents the neuron that the weight is connected to in layer l, and k represents the neuron that the weight is connected to from layer $l-1$. Further, the vector of activations at layer l is given by a^l, and the vector of biases at layer l is given by b^l. Note that

n_l is the number of neurons in layer l, and n_{l-1} is the number of neurons in layer $l-1$:

$$W^l = \begin{bmatrix} w^l_{11} & w^l_{12} & \cdots & w^l_{1n_{l-1}} \\ w^l_{21} & w^l_{22} & \cdots & w^l_{2n_{l-1}} \\ \vdots & \vdots & \ddots & \vdots \\ w^l_{n_l 1} & w^l_{n_l 2} & \cdots & w^l_{n_l n_{l-1}} \end{bmatrix} \; a^l = \begin{bmatrix} a^l_1 \\ \vdots \\ a^l_{n_l} \end{bmatrix} \; b^l = \begin{bmatrix} b^l_1 \\ \vdots \\ b^l_{n_l} \end{bmatrix}$$

4 Extracting Features through the Forward Pass

Now that we have developed an understanding of the fundamental components that define a neural network, we can begin implementing the forward pass. The **forward pass** is the process by which inputs are successively transformed through each layer of the network using the current weights and biases. In other words, it involves propagating data forward through the network, as illustrated in the previous diagram, to produce a final prediction.

4.1 Neural Networks as a Series of Nested Equations

Every activation a^l_j of the jth neuron in the lth layer is related to activations in the $(l-1)$th layer by the equation:

$$a^l_j = \sigma\left(\sum_{k=1}^{n_{l-1}} w^l_{jk}\, a^{l-1}_k + b^l_j\right), \tag{1}$$

where σ is the elementwise **activation function**, w^l_{jk} is the weight from the neuron k to neuron j, a^{l-1}_k is the activation of the neuron from the previous layer corresponding to the weight, and b^l_j is the bias of neuron j. This equation gives us the value of the activation of any neuron in any layer.

We can equally write the equation in vectorized form as:

$$a^l = \sigma\left(W^l\, a^{l-1} + b^l\right). \tag{2}$$

This equation gives us the vector of activations in any layer l. Notice that equation 1 and 2 are both dependent on the activations and weights of the previous layer. Thus, to find the value at any neuron in the network, we produce a chain of nested equations, yielding the backbone of the forward pass. For example, if we were to combine these equations, we can write the series of nested equations to produce the value of the activation vector for the 4th layer for our example diagram of a neural network (Figures 4 & 6). In order to make the notation simpler, we will define z^l to be the input to the activation function σ at layer l, or $z^l = W^l a^{l-1} + b^1$.

$$
\begin{aligned}
a^4 &= \sigma\left(z^4\right) \\
&= \sigma\left(w^4 a^3 + b^4\right) \\
&= \sigma\left(w^4 \sigma\left(z^3\right) + b^4\right) \\
&= \sigma\left(w^4 \sigma\left(w^3 a^2 + b^3\right) + b^4\right) \\
&= \sigma\left(w^4 \sigma\left(w^3 \sigma\left(z^2\right) + b^3\right) + b^4\right) \\
&= \sigma\left(w^4 \sigma\left(w^3 \sigma\left(w^2 a^1 + b^2\right) + b^3\right) + b^4\right).
\end{aligned}
$$

Note that a^1 simply represents the input data. Thus we have derived our series of nested equations constituting the forward pass resulting in the activation vector in the 4th or last layer of our example.

4.2 Importance of σ

The term σ represents the **activation function**, which provides multiple functionality for us. First, it adds non-linearity to the data—allowing us to separate non-linearly separable data, such as images or speech. Second, it helps to activate neurons, or decide which neurons to fire. This is an important part of feature extraction, as it decides which information to pass forward in our

network. An extremely low activation will not put much importance on a feature in separating the data as a higher activation would. Finally, the activation function can enable complex decision boundaries, such as curves, which are necessary for classifying complex data.

There are many possible activation functions to choose from, including ReLU, sigmoid, and multiple others. Each one can be interpreted differently, and has varying results based on the type of problem you are trying to solve with your network. There has been some research done on these, and certain functions are well known to be better at specific tasks than others.

4.3 Bringing it All Together

At each layer, we have a transformation of the input by the weight matrix and bias $\sigma(Wa + b)$. Further, weights and biases are our trainable parameters that we will use to approximate the function, and they are initialized randomly. In the next few chapters, we will discuss how we learn from our predictions, calculate the cost, and adjust these parameters for better results.

5 Training the Model: Beginning the Backward Pass with Cost

The backward pass consists of three primary steps. First, we compute the cost by using a **cost function** to measure the discrepancy between the predicted values and the ground truth labels. Next, we compute the gradients via **backpropagation** to determine how each weight and bias contributes to the overall error. Finally, we update the trainable parameters (weights and biases) in order to minimize the cost, typically using **gradient descent**, where parameters are adjusted iteratively over many **epochs**.

Definition 5.1. An **epoch** in machine learning refers to one complete pass through the entire dataset. In practice, this pass is often divided into smaller iterations over batches of the data.

It is important to note that the description above corresponds to the process for a single input (e.g., one image). In practice, both the forward and backward passes are performed over the entire

dataset repeatedly for many epochs. Although various optimization techniques exist to improve efficiency, the total number of computations involved can be extremely large.

Definition 5.2. Learning in a neural network is the process of iteratively updating the weights and biases in order to improve the accuracy of the network's predictions and minimize the cost function.

To begin the backward pass, we first compute the cost at the output layer, which measures how far the predictions deviate from the ground truth values. Since the cost function will be used extensively in the derivation of backpropagation, we denote it as follows. Let L denote the final layer, and let a^L represent the vector of activations at that layer (i.e., the predicted outputs $\hat{y}$):

$$C = C(a^L) = C(\hat{y}). \tag{3}$$

5.1 Understanding Cost Function & Activation Function Pairings

Similarly to the activation function, the cost function can be customized. However, research has been done which indicates that certain combinations lead to better performance. Because backpropagation requires the computation of many partial derivatives, it is advantageous to select cost functions whose gradients are simple and computationally efficient. In practice, the choice of cost function is guided by the type of problem being solved (e.g., classification or regression) as well as the activation function used in the final layer of the network.

Example 5.3. In a binary classification task, the activation function in the last layer is the **sigmoid** activation and the cost function is the **binary cross-entropy loss** function. Their definitions are stated as follows, where $C(y, \hat{y})$ is the equation for binary cross-entropy, y represents the ground truth labels of the data, $\hat{y}$ represents the predicted values from the output, and the equation to

derive $\hat{y}$ comes from the output of the last layer (our predictions) as a function of the sigmoid activation:

$$C(y, \hat{y}) = -y \ln \hat{y} - (1 - y) \ln(1 - \hat{y}) \tag{4}$$

$$\hat{y} = a^L = \sigma\left(z^L\right) = \frac{1}{1 + e^{-z^L}}. \tag{5}$$

Here, it is important to note that the formula for binary cross-entropy is closely related to the concept of maximum likelihood estimation in statistics. In particular, the expression corresponds to the negative log-likelihood function of estimating a Bernoulli distribution parameter. While a full treatment of this connection is beyond the scope of this paper, it is useful to recognize that the effectiveness of this cost function is grounded in its statistical interpretation and its derivation from probability.

6 Continuing the Learning Process with Backpropagation

To calculate the error of the output, we need to calculate the gradients of the cost function to understand how changing any one of the weights or biases changes the output of the cost function (thus improving our predictions). Therefore, we utilize the chain rule to propagate backwards through layers.

Intuitively, each activation is based on the activations, weights, and bias of the previous layer. Thus, to find how the current weight or bias affects the output of the network, we need to determine how the weight or bias affects the next layer in the network. The main idea is that since our network consists of nested functions, the gradient of the last layer in our function depends on the result of other functions. Therefore, we must use the chain rule, since it tells us how to compute the derivative of a function that depends on other functions.

Further, the definition of the error from the output layer is given as:

$$\delta^L = \frac{\partial C}{\partial z^L},$$

where δ^L is the error at layer L (the output layer), C is the cost function, and z^L is the pre-activation to layer L. Note that we take the derivative with respect to z^L (rather than directly with respect to a^L, the activation function of the last layer L which yields the predictions of the final layer) because z^L is the variable representing the output of the previous layer that propagates forward through the network. In general, we define the error at any layer l as $\delta^l = \frac{\partial C}{\partial z^l}$. Expressing the error in terms of z^L makes it possible to systematically propagate gradients backward through each layer.

The **backpropagation algorithm** can be summarized by four key equations:

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \tag{BP1}$$

$$\delta^l = \left((W^{l+1})^T \delta^{l+1}\right) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

Note that BP1 and BP2 are in vector notation, while BP3 and BP4 are denoted component-wise. This distinction is important to note for the following sections, where we will examine each equation in detail.

6.1 Equation BP1

The purpose of the first equation is to compute the error in the output layer. This step is essential because, in order to determine how any individual weight or bias contributes to the total cost, we

must first compute the error at its corresponding layer. Since backpropagation proceeds from the output layer backward, we begin by computing the error at the final layer.

In BP1, $\nabla_a C$ denotes the gradient of the cost function with respect to the activation vector a^L in the output layer, $\odot$ represents the Hadamard (element-wise) product, and $\sigma'(z^L)$ is the (element-wise) derivative of the activation function evaluated at the pre-activation input z^L.

As stated previously, the error at the output layer by definition is

$$\delta^L = \frac{\partial C}{\partial z^L}.$$

Since the cost C depends on z^L through the activation $a^L = \sigma(z^L)$, we apply the chain rule:

$$\frac{\partial C}{\partial z^L} = \frac{\partial C}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L}.$$

Here, $\frac{\partial C}{\partial a^L}$ is precisely $\nabla_a C$, and $\frac{\partial a^L}{\partial z^L}$ is the derivative of the activation function, $\sigma'(z^L)$. Because these quantities are vectors, the multiplication is performed element-wise, yielding

$$\delta^L = \nabla_a C \odot \sigma'(z^L). \tag{BP1}$$

6.2 Equation BP2

The second backpropagation equation is a key component of the algorithm. It allows us to express the error δ^l at any layer l in terms of the error at the next layer $l+1$. Once we compute the error at the output layer using BP1, we can recursively apply the relationship in BP2 to determine the error at every preceding layer in the network.

This backward propagation of error is crucial because it enables us to compute how each weight and bias contributes to the total cost. For reference, BP2 is given by:

$$\delta^l = \frac{\partial C}{\partial z^l} = \left((W^{l+1})^T \delta^{l+1}\right) \odot \sigma'(z^l) \tag{BP2}$$

Here, $\delta^l = \frac{\partial C}{\partial z^l}$ represents the error at layer l, i.e., the derivative of the cost C with respect to the pre-activation input z^l. The term $(W^{l+1})^T$ is the transpose of the weight matrix for layer $l+1$, δ^{l+1} is the error at the next layer, $\odot$ denotes element-wise (Hadamard) multiplication, and $\sigma'(z^l)$ is the derivative of the activation function with respect to z^l.

Although the full derivation relies on a careful application of the chain rule through the error of the layers in the network, the structure of BP2 is intuitive. The error at layer l depends on how errors propagate backward from layer $l+1$, weighted by the connections between the layers. This error is then influenced by how sensitive the activation function is at layer l, which is captured by $\sigma'(z^l)$.

To better understand this, note that the cost C depends on z^l through the activation $a^l = \sigma(z^l)$, and a^l in turn influences the next layer:

$$z^{l+1} = W^{l+1}a^l + b^{l+1}. \tag{6}$$

Thus, changes in z^l affect z^{l+1}, and ultimately the cost C, making it natural to apply the chain rule across layers.

Consider the j-th neuron in layer l. Applying the chain rule through layer $l+1$, we obtain

$$\begin{aligned}\delta_j^l &= \frac{\partial C}{\partial z_j^l} \\ &= \sum_{k=1}^{n_{l+1}} = \frac{\partial C}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}.\end{aligned}$$

This expression shows that the error at neuron j in layer l is computed by summing over all neurons k in layer $l+1$, accounting for both their contribution to the cost and their dependence on z_j^l. Since $\frac{\partial C}{\partial z_k^{l+1}} = \delta_k^{l+1}$, we rewrite the expression as

$$\delta_j^l = \sum_k \delta_k^{l+1} \frac{\partial z_k^{l+1}}{\partial z_j^l}.$$

To evaluate the remaining term, recall that

$$z_k^{l+1} = \sum_{m=1}^{n_l} w_{km}^{l+1}\, \sigma(z_m^l) + b_k^{l+1}.$$

Taking the partial derivative with respect to z_j^l, all terms with $m \neq j$ vanish, leaving

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{kj}^{l+1}\, \sigma'(z_j^l).$$

Substituting this back into the expression for δ_j^l, we obtain

$$\delta_j^l = \sum_k \delta_k^{l+1}\, w_{kj}^{l+1}\, \sigma'(z_j^l).$$

Finally, writing this in vector form yields

$$\delta^l = \left((W^{l+1})^T \delta^{l+1}\right) \odot \sigma'(z^l), \tag{BP2}$$

which is the desired backpropagation equation (BP2).

6.3 Equation BP3

The third backpropagation equation is where we bridge the error of the network to any bias of the network. The purpose of the equation is thus to give us an equation for the rate of change of the cost with respect to any bias in the network. Here, $\frac{\partial C}{\partial b_j^l}$ is simply the change in cost of the network with respect to the bias b of layer l at neuron j. As we will show, it is given by δ_j^l, which is the error of layer l at neuron j.

Deriving this equation is relatively straightforward, but it is helpful to examine each step carefully. Our goal is to compute the partial derivative of the cost function C with respect to the bias term b_j^l in layer l. To do this, we note that the bias b_j^l does not affect the cost C directly, but instead influences it through the intermediate variable z_j^l, which is the input to the activation function at neuron j in layer l. Recall that

$$z_j^l = \mathrm{row}_j(W^l) \cdot a^{l-1} + b_j^l,$$

so z_j^l depends explicitly on b_j^l.

Because of this dependency, we apply the chain rule to express $\frac{\partial C}{\partial b_j^l}$ in terms of $\frac{\partial C}{\partial z_j^l}$:

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \cdot \frac{\partial z_j^l}{\partial b_j^l}.$$

We now evaluate the second term, $\frac{\partial z_j^l}{\partial b_j^l}$. Now, from the definition of z_j^l, we see that b_j^l is a simple additive term (it is not a function of anything). Therefore, taking the derivative of z_j^l with respect to b_j^l is

$$\frac{\partial z_j^l}{\partial b_j^l} = 1.$$

Then, we can substitute this result back into the chain rule expression, which gives

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l}.$$

Finally, by definition, the quantity δ_j^l represents the error at neuron j in layer l, which is given by

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}.$$

Thus, we arrive at the result:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l. \tag{BP3}$$

6.4 Equation BP4

The purpose of equation BP4 is to provide an expression for the rate of change of the cost function with respect to any weight in the network, given by $\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1}\delta_j^l$. Specifically, w_{jk}^l denotes the weight in layer l connecting neuron k in layer $l-1$ to neuron j in layer l, a_k^{l-1} is the activation of neuron k in the previous layer, and δ_j^l is the error at neuron j in layer l. This equation can be derived in a manner similar to BP3.

To compute the partial derivative of C with respect to the weight w^l_{jk}, we again consider the pre-activation term z^l_j, where

$$z^l_j = \text{row}_j(W^l) \cdot a^{l-1} + b^l_j.$$

Applying the chain rule, we obtain:

$$\frac{\partial C}{\partial w^l_{jk}} = \frac{\partial C}{\partial z^l_j} \cdot \frac{\partial z^l_j}{\partial w^l_{jk}}.$$

The second term is straightforward to compute:

$$\frac{\partial z^l_j}{\partial w^l_{jk}} = a^{l-1}_k,$$

since z^l_j is a linear combination of the activations from the previous layer, the derivative is simply the coefficient on w^l_{jk}. The first term, $\frac{\partial C}{\partial z^l_j}$, is precisely the definition of the error at neuron j in layer l, denoted by δ^l_j. Combining these results, we obtain:

$$\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \, \delta^l_j. \tag{BP4}$$

6.5 Calculating the Gradient of Binary Cross Entropy with Sigmoid

As previously noted in Section 5.1, there are certain pairings we typically make for the cost function and activation function of the last layer, due to the type of problem we are trying to solve, and the elegance in which their derivatives reduce. To show how this process works, we can use the previous example of a binary classification task using binary cross-entropy and sigmoid. For reference, the equations for binary cross-entropy ($C(y, \hat{y})$) and our predicted values as a function of the sigmoid activation in the last layer ($\hat{y}$) are given as follows:

$$C(y, \hat{y}) = -y \ln \hat{y} - (1 - y) \ln(1 - \hat{y}) \tag{4}$$

$$\hat{y} = \sigma\left(z^L\right) = \frac{1}{1 + e^{-z^L}}. \tag{5}$$

Another thing that will be important to note for later on is the complement to $\hat{y}$, $(1-\hat{y})$. Since we know $\hat{y}$, we can derive this simply as follows:

$$1-\hat{y} = 1 - \frac{1}{1+e^{-z^L}} = \frac{e^{-z^L}}{1+e^{-z^L}} \tag{7}$$

Now, we want to find the derivative of the cost with respect to the input to the last layer, or $\frac{\partial C}{\partial z^L}$. As stated previously, in order to derive this, we need to chain through the activation function in the last layer (in this case, we need to chain through the sigmoid function). Another way to put this is that we need to chain through the output of the last layer to the input of the last layer. Thus we have:

$$\frac{\partial C}{\partial z^L} = \frac{\partial C}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^L}. \tag{8}$$

Thus, we must derive $\frac{\partial C}{\partial \hat{y}}$ and $\frac{\partial \hat{y}}{\partial z^L}$ to come up with a formula for the error.

6.5.1 Deriving $\frac{\partial C}{\partial \hat{y}}$

Deriving $\frac{\partial C}{\partial \hat{y}}$ is straightforward. Recall that the binary cross-entropy cost function is given by:

$$C(y, \hat{y}) = -y \ln \hat{y} - (1-y)\ln(1-\hat{y}).$$

Taking the derivative with respect to $\hat{y}$, we obtain:

$$\frac{\partial C}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}.$$

Combining the terms over a common denominator yields:

$$\frac{\partial C}{\partial \hat{y}} = \frac{-y(1-\hat{y}) + \hat{y}(1-y)}{\hat{y}(1-\hat{y})}.$$

Expanding and simplifying the numerator, we obtain:

$$\frac{\partial C}{\partial \hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}.$$

6.5.2 Deriving $\frac{\partial \hat{y}}{\partial z^L}$

We now compute the second component of the chain rule needed to determine the error at the output layer. Recall that for a sigmoid activation function,

$$\hat{y} = \sigma(z^L) = \frac{1}{1 + e^{-z^L}}.$$

Then, taking the derivative with respect to z^L, we obtain:

$$\begin{aligned}\frac{\partial \hat{y}}{\partial z^L} &= \frac{d}{dz^L}\left(\frac{1}{1 + e^{-z^L}}\right) \\ &= \frac{-1}{(1 + e^{-z^L})^2} \cdot \left(-e^{-z^L}\right).\end{aligned}$$

We can simplify this result to get:

$$\frac{\partial \hat{y}}{\partial z^L} = \frac{e^{-z^L}}{(1 + e^{-z^L})^2}.$$

Then, we rewrite the expression by factoring the denominator:

$$\frac{\partial \hat{y}}{\partial z^L} = \left(\frac{e^{-z^L}}{1 + e^{-z^L}}\right) \cdot \left(\frac{1}{1 + e^{-z^L}}\right).$$

Observe that:

$$\frac{1}{1 + e^{-z^L}} = \hat{y} \quad \text{and} \quad \frac{e^{-z^L}}{1 + e^{-z^L}} = 1 - \hat{y}.$$

Substituting these expressions, we obtain:

$$\frac{\partial \hat{y}}{\partial z^L} = (1 - \hat{y})\hat{y}.$$

6.5.3 Deriving the Final Error of the Network

Finally, we can sub in these values to the chain of partial derivatives from (7). This yields:

$$\frac{\partial C}{\partial z^L} = \frac{\hat{y} - y}{\hat{y}\,(1 - \hat{y})} \cdot \hat{y}\,(1 - \hat{y}),$$

which then simplifies to:

$$\frac{\partial C}{\partial z^L} = \hat{y} - y. \tag{9}$$

By pairing sigmoid and binary cross entropy, we were able to reduce the equation for error to simply the difference between the predicted and ground truth values. This offers a simple and intuitive way to interpret the error of the network, and makes calculating the error at each layer much simpler when applying the equations of backpropagation to calculate the partial derivatives with respect to each weight and bias throughout the network. In addition, it is important to note that similar simplifications happen for other pairings, such as softmax and categorical cross entropy.

7 Gradient Descent: the Update Process

7.1 Motivation

After we have calculated the error of the network (thus calculating the gradients of the cost at every layer due to every weight and bias), we need to update the weights and biases based on our learned information in order to minimize the cost function. This is done iteratively over many epochs by moving parameters in the direction that reduces the cost, or the opposite direction of the gradient. To build intuition, consider a three-dimensional landscape of peaks and valleys representing the cost function. While the true cost function exists in a much higher-dimensional space, a 3D visualization provides a useful conceptual model. At the start of training, when the weights and biases are randomly initialized, the network's error is typically high, placing us near the top of one of these peaks. Our ultimate goal is to descend into a minimum (ideally the global minimum) to minimize the cost of our network, thus resulting in optimal performance for prediction. However, it is very computationally complex to calculate this entire landscape at once and descend to the global minimum immediately (especially with a large neural network with multiple parameters) so we must take an iterative approach to step the landscape and find the

minimal cost.

7.2 Defining Gradient Descent

Recall that the gradient of a function points in the direction of its steepest increase. Therefore, to reduce the value of the cost function, we update the parameters in the direction opposite to the gradient. For each trainable parameter w and b, we update them iteratively according to these rules:

$$w := w - \eta \cdot \frac{\partial C}{\partial w} \tag{GD1}$$

$$b := b - \eta \cdot \frac{\partial C}{\partial b} \tag{GD2}$$

where b is a given bias in the network, w is a given weight in the network, and η is the learning rate, which denotes the step size of the gradient descent. Note that while we update parameters every iteration, $\frac{\partial C}{\partial w}$ and $\frac{\partial C}{\partial b}$ represent the average cost with respect to the weights and biases of all of the data in the batch being processed.

7.3 Improving Gradient Descent

In Chapter 5, we briefly introduced the idea of *batching* when defining an *epoch*. A **batch** refers to a subset of the training data used to compute gradients during optimization. Performing gradient descent over the entire dataset at every iteration—known as **batch gradient descent**—can be computationally expensive, especially for large datasets.

To address this, **mini-batch gradient descent** is often used, where gradients are computed using smaller subsets of the data (or batches). Although this approach produces less precise gradient estimates compared to using the full dataset, it significantly reduces computation time and typically leads to faster overall convergence in practice.

Another variation is **stochastic gradient descent** (SGD), which updates parameters using only a single randomly selected data point at each step. While SGD introduces even more variance into the gradient estimates, it can be useful for escaping local minima and can be highly efficient for very large datasets.

Among these approaches, mini-batch gradient descent is the most commonly used, as it provides a practical balance between computational efficiency and the accuracy of gradient estimates.

8 Application: Using a Neural Network for Image Classification

While it is important to understand the mathematical foundations of neural networks in order to explain how data is transformed into predictions, theory alone is not sufficient for practical use. The widespread adoption of neural networks became feasible only with the development of GPUs, which significantly accelerated the computational processes required for training such models.

In the Appendix, we present an implementation of a neural network built from scratch in Python, using a flower image dataset provided by the TensorFlow framework. The objective is to classify images into one of five categories: dandelion (898 samples), daisy (633 samples), tulips (799 samples), sunflowers (699 samples), and roses (641 samples). Although the dataset is relatively small, it is reasonably balanced across classes; therefore, classification accuracy is used as the primary evaluation metric.

Prior to splitting the data into training and testing sets, the images are preprocessed to ensure consistency and improve model performance. All images are resized to 32×32 pixels, as the network requires inputs of uniform dimensions. Additionally, pixel values are scaled within the range $[0, 1]$, which helps stabilize training by preventing excessively large gradients. Although these transformed values no longer correspond directly to raw pixel intensities, they provide meaningful feature representations for the network.

The input layer consists of 3072 neurons, corresponding to flattened 32×32 RGB images

$(32 \times 32 \times 3 = 3072)$. The network includes two hidden layers with 128 and 64 neurons, respectively, followed by an output layer with 5 neurons, one for each flower class. The number of neurons in the hidden layers is chosen heuristically.

The network uses the ReLU activation function in the hidden layers and the Softmax activation function in the output layer. These activation functions are implemented in code and applied during the forward pass. During this process, intermediate values—such as inputs and activations—are stored at each layer to facilitate gradient computation during backpropagation. The cost is computed using the categorical cross-entropy loss function, which is commonly paired with Softmax for multi-class classification tasks.

Training is performed using mini-batch gradient descent to improve computational efficiency. The cost is recorded every 10 epochs to monitor how it decreases as the network learns.

Evaluation on the test set yields an accuracy of 44.28%. While this indicates relatively poor performance, it could likely be improved by increasing the number of hidden layers, tuning hyperparameters, or using higher-resolution images. For the purposes of this project, however, the model serves primarily as a demonstration of how neural networks operate in practice.

9 Conclusion

Neural networks provide a powerful framework for modeling complex, high-dimensional, and non-linear data, making them particularly well-suited for tasks such as image classification. As a result, they play a key role in real-world applications, including recommendation systems. In this project, we developed a comprehensive understanding of how these models function by examining both their mathematical foundations and their computational implementation. We began by exploring how data is represented using tensors, establishing the structure required to efficiently process image data. We then examined the fundamentals of the forward pass, including its formulation as a series of nested equations and the critical role of activation functions in introducing non-linearity.

Building on this, we analyzed the backward pass, including the computation of the cost function, the propagation of error through backpropagation, and the optimization of model parameters using gradient descent. Finally, to connect theory with practice, we implemented a neural network from scratch in Python and applied it to the task of classifying flower images. Although the model achieved modest accuracy, the implementation provided valuable insight into how neural networks learn from data, adjust their parameters through optimization, and ultimately approximate complex functions. Overall, this paper establishes a strong foundation in the mathematical principles underlying neural networks and demonstrates how these principles translate into practical tools for image classification.

A Appendix: Definitions and Notation

A.1 Key Definitions

- **Activation Function:** A mathematical operation applied to the neuron's output to introduce non-linearity, enabling the network to learn complex patterns.
- **Bias:** An additive constant that allows a neuron to shift its activation function left or right, helping the model better fit the data.
- **Cost Function:** A function that measures the difference between predicted outputs and true labels.
- **Epoch (Definition 5.1):** An epoch in machine learning represents one entire pass through the dataset, and can be split into iterations over batches of the dataset.
- **Learning (Definition 5.2):** Learning in a neural network is the process of updating the weights and bias to improve the outcome of the network and minimize the cost.
- **Tensor (Definition 2.1):** In machine learning, a tensor is a multi-dimensional array defined by giving its rank, data type, and shape.
- **Weight:** A learnable parameter that acts as a "volume knob," scaling an input to determine its relative importance or influence on the network's final prediction.

A.2 Notation

a_j^l : Activation of neuron j in layer l

z_j^l : Input to neuron j in layer l

W^l : Weight matrix for layer l

w_{jk}^l : Weight connecting neuron k in layer $l-1$ to neuron j in layer l

b^l : Bias vector for layer l

b_j^l : Bias of neuron j in layer l

σ : Activation function

C : Cost function

$\nabla_a C$: Gradient of the cost with respect to activations

δ^l : Error vector of layer l

δ_j^l : Error of neuron j in layer l

η : Learning rate

$\odot$: Hadamard (element-wise) product

A.3 Core Equations

Forward Pass:

$$a^l = \sigma(W^l a^{l-1} + b^l)$$

Backpropagation:

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \tag{BP1}$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

Binary Cross Entropy:

$$C(y, \hat{y}) = -y \ln y - (1 - y) \ln(1 - \hat{y})$$

Gradient Descent Updates:

$$w := w - \eta \frac{\partial C}{\partial w} \tag{GD1}$$

$$b := b - \eta \frac{\partial C}{\partial b} \tag{GD2}$$

Predicted Values from a Network as a Function of Sigmoid:

$$\hat{y} = \sigma\left(z^2\right) = \frac{1}{1 + e^{-z^L}}$$

Appendix (Continued)

A4: Load Libraries

```
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import tensorflow as tf
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

A5: Load & Visualize the Flowers dataset

```
# Load the dataset from tensor flow data sets (tfds)
# Our dataset is'tf_flowers' -> a built-in dataset in TensorFlow Dataset
# as_supervised=True -> returns data as (image, label) tuples instead of
# with_info=True -> also returns metadata (info) about the dataset
dataset, info = tfds.load('tf_flowers', as_supervised=True, with_info=Tr

# Extract the training portion of the dataset
# The tf_flowers dataset only has a 'train' split (no separate test set
train_data = dataset['train']

# Get class names from the dataset metadata
# info.features -> describes the structure of the dataset
# 'label' -> target variable (flower type)
# .names -> list of class names (e.g., ['daisy', 'dandelion', ...])
class_names = info.features['label'].names

# Create a figure to display images; figsize=(8,8) sets the width and he
plt.figure(figsize=(8,8))

# Loop through the first 9 examples in the training dataset to display
# enumerate() -> give both index (i) and the data (image, label)
# .take(9) -> limit the dataset to the first 9 samples
for i, (image, label) in enumerate(train_data.take(9)):
    # Create a subplot grid of 3 rows × 3 columns; i+1 specifies the pos
    plt.subplot(3, 3, i+1)

    # Display the image -> 'image' is a tensor (multi-dimensional array)
    plt.imshow(image)
```

```
    # Set the title of the subplot using the label, which is an integer,
    plt.title(class_names[label])

    # Turn off axis ticks and labels
    plt.axis('off')

# Display
plt.show()
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/tf_flowers/3.0.1 ha
Downloading and preparing dataset Unknown size (download: Unknown size, g
```

Dl Completed...: 100% 1/1 [00:02<00:00, 2.34s/ url]

Dl Size...: 100% 218/218 [00:02<00:00, 93.52 MiB/s]

```
Dataset tf_flowers downloaded and prepared to /root/tensorflow_datasets/t
```

A6a: Preprocessing

We will simplify the input for the neural network by reducing the dimensionality (image size becomes 32x32, and we scale the pixel values to be between [0, 1] to make a more easily interpretted and controlled range of values for the neural network). This is important because it reduces the "curse of dimensionality" - increasing model performance, computational effeciency, and generalization. Resizing the images also serves a dual purpose; the neural network expects our images to be a certain size when inputted to the network.

```
# First preprocess each image
def preprocess(image, label):

    # Resize image to 32x32 pixels -> the original images in tf_flowers
    image = tf.image.resize(image, (32, 32))

    # Ensure pixel values are float32 & scale them
    # tf.cast -> converts from integer (0–255) to float. Then divide by
    # Note: this is a simple scaling method, so we can scale on the enti
    image = tf.cast(image, tf.float32) / 255.0

    # Flatten the image into a 1D vector -> 32 * 32 * 3 (RGB channels) =
    # [-1] -> flatten everything into one dimension
    image = tf.reshape(image, [-1])

    # Convert TensorFlow tensors (what TensorFlow gives you the data as)
    return image.numpy(), label.numpy()

# Once preprocessed, we need to build arrays so we can represent each im
# Apply preprocessing to every (image, label) pair in the dataset
# This creates a list of tuples: [(image_array, label), ...]
data = [preprocess(img, lbl) for img, lbl in train_data]

# Extract all image vectors into a single NumPy array
# Each d[0] is a flattened image (length 3072; 32x32x3). Result shape: (
X = np.array([d[0] for d in data])

# Extract all labels into a NumPy array
# Each d[1] is a single integer label. Result shape: (number_of_samples,
y = np.array([d[1] for d in data])

# Print shapes to verify
print("X shape:", X.shape)
print("y shape:", y.shape)

# Split the data into training and test sets -> this is important for ev
# Shuffle the dataset before splitting to prevent bias from ordered data
```

```
indices = np.random.permutation(len(X))

# Apply shuffled indices to X and y
X = X[indices]
y = y[indices]

# Define split (80% training, 20% testing)
split = int(0.8 * len(X))

# Split the dataset. The training data will be used for learning
X_train = X[:split]
y_train = y[:split]

# The test data will only be used for evaluation
X_test = X[split:]
y_test = y[split:]

# Print shapes to confirm split
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X shape: (3670, 3072)
y shape: (3670,)
X_train shape: (2936, 3072)
X_test shape: (734, 3072)
```

We can look at some statistics of our dataset, to get a better idea about its distribution and balance.

```
print("--- Dataset Statistics ---")

print(f"Total number of samples: {len(X)}")

num_classes = len(class_names)
print(f"Number of classes: {num_classes}")

# Class Distribution (Counts per category)
print("\nClass Distribution:")
unique_labels, counts = np.unique(y, return_counts=True) # Retreive the
for label, count in zip(unique_labels, counts): # For each label and cou
    print(f"  {class_names[label]}: {count} samples")

# Image Dimensions after preprocessing (before flattening)
# Assuming the original resize was to 32x32 pixels with 3 color channels
print(f"\nPreprocessed image dimensions (H, W, C): (32, 32, 3)")
print(f"Flattened feature vector size (32*32*3): {X.shape[1]}")

# Data Types
print(f"\nFeature array (X) data type: {X.dtype}")
```

```
print(f"Label array (y) data type: {y.dtype}")
```

```
--- Dataset Statistics ---
Total number of samples: 3670
Number of classes: 5

Class Distribution:
  dandelion: 898 samples
  daisy: 633 samples
  tulips: 799 samples
  sunflowers: 699 samples
  roses: 641 samples

Preprocessed image dimensions (H, W, C): (32, 32, 3)
Flattened feature vector size (32*32*3): 3072

Feature array (X) data type: float32
Label array (y) data type: int64
```

A6b: Visualize After Preprocessing

```
# Create a figure for plotting
plt.figure(figsize=(10, 4))

# Loop through the first 5 samples
for i in range(9):

    # Get flattened image vector (length 3072)
    img = X[i]

    # Reshape back to image format (32x32x3) -> otherwise the images are
    # This will only change them for viewing after preprocessing, not fo
    img = img.reshape(32, 32, 3)

    # Create subplot (3 row, 3 columns, i + 1 specifies position)
    plt.subplot(3, 3, i + 1)

    # Display the image
    plt.imshow(img)

    # Add class label as title
    plt.title(class_names[y[i]])

    # Hide axis for cleaner output
    plt.axis('off')

# Fix layout spacing
plt.tight_layout()
```

```
# Display
plt.show()
```

A7: Build NN From Scratch - Initialize Parameters

Initialize parameters:

We will have input 3072 (32x32x3) neurons, hidden layers with 128 and 64 neurons, and output 5 neurons (5 flower classes).

```
# Set a random seed for reproducibility -> this way every time the code
# we get the same random numbers (same weights)
np.random.seed(42)

# Layer 1 -> Input Layer to Hidden Layer 1
# Initialize weights for first layer -> shape: (3072, 128)
# 3072 -> number of input features (flattened image: 32*32*3)
# 128 -> number of neurons in first hidden layer
# np.random.randn(...) generates values from a normal distribution (mean
W1 = np.random.randn(3072, 128) * np.sqrt(2 / 3072)

# Initialize biases for first layer -> shape: (1, 128) -> one bias per n
b1 = np.zeros((1, 128)) # Initialized to zeros

# Layer 2 -> Hidden Layer 1 to Hidden Layer 2 -> Shape: (128, 64)
# Initialize weights for second layer
```

```
# 128 -> input from previous layer
# 64 -> number of neurons in second hidden layer
# Small random values (0.01 scaling) to prevent large activations early
W2 = np.random.randn(128, 64) * np.sqrt(2 / 128)

# Initialize biases for second layer. Shape: (1, 64)
b2 = np.zeros((1, 64))

# Layer 3 -> Hidden Layer 2 to Output Layer -> Shape: (64, 5)
# Initialize weights for output layer
# 64 -> input from second hidden layer
# 5 -> number of output classes (flowers: 5 types)
W3 = np.random.randn(64, 5) * np.sqrt(2 / 64)

# Initialize biases for output layer. Shape: (1, 5)
b3 = np.zeros((1, 5))

# Print shapes to verify
print("W1 shape:", W1.shape)
print("b1 shape:", b1.shape)

print("W2 shape:", W2.shape)
print("b2 shape:", b2.shape)

print("W3 shape:", W3.shape)
print("b3 shape:", b3.shape)
```

```
W1 shape: (3072, 128)
b1 shape: (1, 128)
W2 shape: (128, 64)
b2 shape: (1, 64)
W3 shape: (64, 5)
b3 shape: (1, 5)
```

A8: Forward Pass

Now we implement the forward pass, including the activation functions (ReLU, and Softmax) and the forward propagation through each of the layers. The implemented relu and softmax functions are simply called when we need to perform the respective activation function. The forward function simply implements the chain of nested equations we perform and stores the values of Z and A (the inputs and activations at each level) in a variable to be used for the backward pass later on.

```
# ReLU Activation Function
def relu(Z):
    # Apply element-wise ReLU
    # For each value in Z:
    #   if value > 0 -> keep it
```

```
    #   if value <= 0 -> replace with 0
    # np.maximum compares each element with 0
    return np.maximum(0, Z)


# Softmax Activation Function
def softmax(Z):
    # Subtract the maximum value in each row for numerical stability
    # axis=1 -> operate across columns (per sample)
    # keepdims=True -> keeps shape compatible for broadcasting
    # This prevents overflow when applying exponential
    expZ = np.exp(Z - np.max(Z, axis=1, keepdims=True))

    # Compute softmax probabilities
    # Divide each exponential by the sum of exponentials in that row
    # Result: each row sums to 1 (valid probability distribution)
    return expZ / np.sum(expZ, axis=1, keepdims=True)


# Forward Pass
def forward(X):

    # Layer 1 -> Input to Hidden Layer 1
    # Linear transformation:
    # Z1 = XW1 + b1
    # X shape:  (batch_size, 3072)
    # W1 shape: (3072, 128)
    # Result:   (batch_size, 128)
    Z1 = X @ W1 + b1

    # Apply ReLU activation: converts linear output into non-linear repr
    # Shape remains (batch_size, 128)
    A1 = relu(Z1)

    # Layer 2 -> Hidden Layer 1 to Hidden Layer 2
    # Linear transformation:
    # Z2 = A1W2 + b2
    # A1 shape: (batch_size, 128)
    # W2 shape: (128, 64)
    # Result:   (batch_size, 64)
    Z2 = A1 @ W2 + b2

    # Apply ReLU activation again. Adds non-linearity so model can learn
    A2 = relu(Z2)

    # Layer 3 -> Hidden Layer 2 to Output Layer
    # Linear transformation:
    # Z3 = A2W3 + b3
    # A2 shape: (batch_size, 64)
    # W3 shape: (64, 5)
    # Result:   (batch_size, 5)
```

```
    Z3 = A2 @ W3 + b3

    # Apply softmax activation. Converts raw scores (logits) into probab
    # Each row will sum to 1
    A3 = softmax(Z3)

    # Store intermediate values for later use (backpropagation). These w
    cache = (Z1, A1, Z2, A2, Z3, A3)

    # A3 -> final predictions (probabilities)
    # cache -> saved values for training (backprop)
    return A3, cache
```

A9: Loss - Cross Entropy

The compute_loss function uses categorical cross entropy, which is adapted from negative log likelihood. We take the negative log likelihoods of the correct classes to predict the loss.

```
def compute_loss(A3, y):

    # Get number of samples (m = number of training examples)
    m = y.shape[0] # y is shape: (m,)

    # Compute the negative log likelihood for each sample
    # A3 is the model's predicted probabilities (after softmax) from the
    # Shape: (m, num_classes)
    #
    # A3[range(m), y] does:
    #   - For each example i, select the probability that model assigned
    #   - range(m) = [0, 1]
    #
    # Example:
    # A3 = [[0.1, 0.7, 0.2], -> sample 0
    #       [0.8, 0.1, 0.1]] -> sample 1
    # y  = [1, 0] -> sample 0 correct class is 1, sample 1 correct class
    #
    # A3[range(2), y] -> [0.7, 0.8] -> find the probability that the mod
    #
    # A3[0, 1] -> row 0, col 1 of A3 -> 0.7. The model assigned a 0.7 pr
    # A3[1, 0] -> row 1, col 0 of A3 -> 0.8. The model assigned a 0.8 pr
    #
    # Then:
    # -np.log(...) converts them into loss values
    # High probability -> low loss
    # Low probability -> high loss
    log_likelihood = -np.log(A3[range(m), y])
```

```
    # Compute average loss across all samples
    # np.sum(...) adds up all individual losses
    # Divide by m -> mean loss
    return np.sum(log_likelihood) / m
```

A10: Backpropagation

```
# Derivative of ReLU activation function
def relu_derivative(Z):
    # Returns True (1) where Z > 0, else False (0)
    # This represents the derivative of ReLU:
    # ReLU'(Z) = 1 if Z > 0, else 0
    return Z > 0


# Backpropagation
def backward(X, y, cache):

    # Unpack stored values from forward pass
    # These were saved to avoid recomputing them
    Z1, A1, Z2, A2, Z3, A3 = cache

    # Number of training examples
    m = X.shape[0]

    # Layer 3 -> output layer ---
    # Copy A3 so we don't modify original predictions
    # A3 = softmax output (probabilities)
    dZ3 = A3.copy()

    # Subtract 1 from the correct class probabilities
    # This is the gradient of softmax + cross-entropy loss
    # For each example i:
    #   dZ3[i, y[i]] -= 1
    dZ3[range(m), y] -= 1

    # Divide by m to average gradients over all samples
    dZ3 /= m

    # Compute gradient of weights for layer 3
    # A2.T shape: (64, m)
    # dZ3 shape:  (m, 5)
    # Result:     (64, 5)
    dW3 = A2.T @ dZ3

    # Compute gradient of biases for layer 3
    # Sum over all samples (axis=0 -> column-wise)
    # keepdims=True keeps shape as (1, 5)
    db3 = np.sum(dZ3, axis=0, keepdims=True)
```

```
    # Layer 2 ---
    # Backpropagate gradient to previous layer
    # W3.T shape: (5, 64)
    # dZ3 shape:  (m, 5)
    # Result:     (m, 64)
    dA2 = dZ3 @ W3.T

    # Apply derivative of ReLU
    # Only keep gradients where Z2 > 0
    # (ReLU blocks negative values)
    dZ2 = dA2 * (Z2 > 0)

    # Gradient of weights for layer 2
    # A1.T shape: (128, m)
    # dZ2 shape:  (m, 64)
    # Result:     (128, 64)
    dW2 = A1.T @ dZ2

    # Gradient of biases for layer 2
    db2 = np.sum(dZ2, axis=0, keepdims=True)


    # Layer 1 ---
    # Backpropagate gradient further back
    # W2.T shape: (64, 128)
    # dZ2 shape:  (m, 64)
    # Result:     (m, 128)
    dA1 = dZ2 @ W2.T

    # Apply derivative of ReLU again
    dZ1 = dA1 * (Z1 > 0)

    # Gradient of weights for layer 1
    # X.T shape:  (3072, m)
    # dZ1 shape: (m, 128)
    # Result:    (3072, 128)
    dW1 = X.T @ dZ1

    # Gradient of biases for layer 1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    # Return all gradients - these will be used to update weights during
    return dW1, db1, dW2, db2, dW3, db3
```

A11: Gradient Descent

```
# Function to create mini-batches
def get_batches(X, y, batch_size=32):
```

```
    # Create a random permutation of indices from 0 to len(X)-1
    # This shuffles the dataset so batches are different each epoch
    # Prevents the model from learning patterns based on data order
    indices = np.random.permutation(len(X))

    # Loop over the dataset in steps of batch_size
    # range(start, stop, step)
    # Example: range(0, 1000, 32)
    for i in range(0, len(X), batch_size):

        # Select a slice of shuffled indices for this batch
        # i:i+batch_size gives a chunk of indices
        batch_idx = indices[i:i+batch_size]

        # Yield (return) the batch of inputs and labels
        # X[batch_idx] -> selects those rows from X
        # y[batch_idx] -> selects corresponding labels
        # 'yield' makes this a generator (memory efficient)
        yield X[batch_idx], y[batch_idx]

# Function to update parameters using gradient descent
def update_params(dW1, db1, dW2, db2, dW3, db3, lr):

    # Declare these variables as global so we can modify them
    # These are the model parameters defined earlier
    global W1, b1, W2, b2, W3, b3

    # Update Layer 1 parameters ---
    # Update weights using gradient descent rule:
    # W = W - learning_rate * gradient
    W1 -= lr * dW1
    # Update biases similarly
    b1 -= lr * db1

    # Update Layer 2 parameters ---
    # Adjust weights for layer 2
    W2 -= lr * dW2
    # Adjust biases for layer 2
    b2 -= lr * db2

    # Update Layer 3 (output layer) parameters ---
    # Update weights for output layer
    W3 -= lr * dW3
    # Update biases for output layer
    b3 -= lr * db3
```

A12: Training Loop

Now we will train our model! Every iteration of the outer `for loop` represents an epoch, and every pass of the inner `for loop` represents a pass through a batch. We will calculate the loss every 10 epochs so that we can monitor how our network minimizes the loss during the training process.

```
# Loop over the dataset multiple times (epochs)
# One epoch = one full pass through the entire dataset
for epoch in range(50):

    # Iterate over mini-batches of data
    # get_batches() returns small chunks of (X, y)
    for X_batch, y_batch in get_batches(X_train, y_train):

        # Forward pass ---
        # Compute predictions (A3) and store intermediate values (cache)
        A3, cache = forward(X_batch)

        # Backward pass ---
        # Compute gradients of weights and biases using backpropagation
        # These gradients tell us how to adjust parameters
        dW1, db1, dW2, db2, dW3, db3 = backward(X_batch, y_batch, cache)

        # Parameter update ---
        # Update weights and biases using gradient descent
        # lr=0.01 is the learning rate (step size)
        update_params(dW1, db1, dW2, db2, dW3, db3, lr=0.01)

    # Monitor training progress ---
    # Every 10 epochs, print the loss
    if epoch % 10 == 0:

        # Calculate loss over the entire dataset to report
        A3_full, _ = forward(X_train)
        # Compute loss every 10th epoch - for testing purposes, not trai
        loss = compute_loss(A3_full, y_train)

        # Print current epoch number and loss value
        # :.4f formats the loss to 4 decimal places
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

# Calculate loss over the entire dataset to report
A3_full, _ = forward(X_train)

# Compute loss for final epoch
loss = compute_loss(A3_full, y_train)
```

```
Epoch 0, Loss: 1.3710
Epoch 10, Loss: 1.0038
Epoch 20, Loss: 1.0081
```

```
Epoch 30, Loss: 0.6216
Epoch 40, Loss: 0.4514
```

A13: Prediction

```
# Function to make predictions
def predict(X):

    # Run a forward pass through the network
    # A3 contains the output probabilities for each class
    # Shape: (num_samples, num_classes)
    # We ignore the cache (_) because we don't need it for prediction
    A3, _ = forward(X)

    # Select the index of the highest probability for each sample
    # axis=1 -> operate across columns (classes)
    # For each row, pick the class with the highest probability
    # Result shape: (num_samples,)
    return np.argmax(A3, axis=1)

# Use the model to make predictions on dataset X_test
# preds will contain predicted class labels (integers)
preds = predict(X_test)

# Compute accuracy
# Compare predictions to true labels (y_test). (preds == y_test) creates
# np.mean(...) computes fraction of correct predictions: True = 1, False
# So mean = (# correct) / (total samples)
# Multiply by 100 -> convert to percentage
accuracy = np.mean(preds == y_test) * 100

print(f"Accuracy: {round(accuracy, 2)}%")
```

```
Accuracy: 44.28%
```

A14: Visualize Predictions

```
# Display the first 9 test images in a 3x3 grid using precomputed predic
plt.figure(figsize=(8, 8))  # Create a square figure

for i in range(9):
    # Take the flattened image and reshape it back to 32x32x3 for displa
    img_display = X_test[i].reshape(32, 32, 3)

    # Create subplot (3 rows, 3 columns, position i+1)
    plt.subplot(3, 3, i + 1)
```

```
        # Display the image
        plt.imshow(img_display)

        # Display the true predictions vs. ground truth class labels
        plt.title(f"Pred: {class_names[preds[i]]}\nTrue: {class_names[int(y_

        # Hide axes for cleaner display
        plt.axis('off')

    # Adjust layout to prevent overlapping
    plt.tight_layout()
    plt.show()
```

A15: Confusion Matrix

This part is extra from our neural network, but is useful in examining the performance of our network as we can see the amount of classes predicted correctly, and if there were any classes that were commonly misclassified as another.

```
# Compute confusion matrix
# confusion_matrix compares true labels (y) vs predicted labels (preds)
# Output is a 2D array where rows = actual classes, and columns = predic
cm = confusion_matrix(y_test, preds)

# Create a figure for the heatmap -> figsize=(6,5) controls width and he
plt.figure(figsize=(6,5))

# Plot the confusion matrix as a heatmap
sns.heatmap(
    cm,                 # The confusion matrix data
    annot=True,         # Show numbers inside each cell
    fmt='d',            # Format numbers as integers (no decimals)

    # Label x-axis (predicted classes) with class names
    xticklabels=class_names,

    # Label y-axis (actual classes) with class names
    yticklabels=class_names
)

# Label the x-axis
# Represents model predictions
plt.xlabel("Predicted")

# Label the y-axis
# Represents true labels (ground truth)
plt.ylabel("Actual")

# Display the plot
```

References

[1] Michael A. Nielsen, "Neural Networks and Deep Learning", *Neural Networks and Deep Learning*, Determination Press, 2015, http://neuralnetworksanddeeplearning.com/.

[2] François Chollet and Matthew Watson, "Deep Learning with Python, Third Edition", *Deep Learning with Python*, Manning Publications, 2025, ISBN 978-1-63343-658-9, https://www.manning.com/books/deep-learning-with-python-third-edition.

[3] Grant Sanderson, "Neural Networks (Deep Learning) — 3Blue1Brown Playlist", *YouTube*, 3Blue1Brown channel, 2017, https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi.